

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

АДЫГЕЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

НОК “Институт точных наук и цифровых технологий”
Кафедра автоматизированных систем обработки информации и
управления

ОТЧЕТ ПО ПРАКТИКЕ

Программная реализация численных методов
Решение нелинейных уравнений

1 курс, группа 1ИВТ

Выполнил:

_____ С. С. Ромащенко
«_____» _____ 2026 г.

Руководитель:

_____ С. В. Теплоухов
«_____» _____ 2026 г.

Майкоп, 2026 г.

1 Введение

Задание 1. Методы решения нелинейных уравнений

Решить уравнение $f(x) = 0$ с точностью до 0,01:

- Вариант 49: $\sqrt{4x+7} = 3 \cos(x)$
- Методом бисекции (деления отрезка пополам) и методом секущих

Задание 2. Методы решения систем линейных алгебраических уравнений

Решить систему линейных уравнений $AX = B$ с точностью до 0,01:

- Вариант 49: метод Крамера
- Матрица $A = \begin{pmatrix} 0,66 & 0,44 & 0,22 \\ 1,54 & 0,74 & 1,54 \\ 1,42 & 1,42 & 0,86 \end{pmatrix}$, вектор $B = \begin{pmatrix} -0,58 \\ -0,32 \\ 0,83 \end{pmatrix}$

2 Ход работы

2.1 Задание 1. Решение нелинейного уравнения

Анализ уравнения

Приведем уравнение $\sqrt{4x+7} = 3\cos(x)$ к виду $f(x) = 0$:

$$f(x) = \sqrt{4x+7} - 3\cos(x) = 0$$

Область определения: $4x + 7 \geq 0 \Rightarrow x \geq -1,75$.

Метод бисекции

Метод основан на теореме Больцано-Коши: если непрерывная функция $f(x)$ принимает на концах отрезка $[a, b]$ значения разных знаков, то внутри отрезка существует корень.

Алгоритм:

1. Вычислить середину отрезка: $c = \frac{a+b}{2}$
2. Если $f(c) = 0$, корень найден
3. Если $f(a) \cdot f(c) < 0$, заменить $b = c$
4. Иначе заменить $a = c$
5. Повторять до достижения точности: $\frac{b-a}{2} < \varepsilon$

Метод секущих

Итерационная формула:

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Условие остановки: $|x_{n+1} - x_n| < \varepsilon$

```
1 import math
2
3 def f(x):
4     """f(x) = (4x+7) - 3cos(x)"""
5     if 4*x + 7 < 0:
6         return None
7     return math.sqrt(4*x + 7) - 3*math.cos(x)
8
9 def bisection(a, b, eps=0.01):
10     if f(a) is None or f(b) is None:
11         return None
12     if f(a) * f(b) > 0:
```

```

13         return None
14
15     n = 0
16     while (b - a) / 2 > eps:
17         c = (a + b) / 2
18         if f(a) * f(c) <= 0:
19             b = c
20         else:
21             a = c
22         n += 1
23     return (a + b) / 2, n
24
25 def secant(x0, x1, eps=0.01):
26     n = 0
27     while n < 1000:
28         f0, f1 = f(x0), f(x1)
29         if f0 is None or f1 is None or abs(f1 - f0) < 1e-12:
30             return None
31         x2 = x1 - f1 * (x1 - x0) / (f1 - f0)
32         if 4 * x2 + 7 < 0:
33             x2 = max(x2, -1.75 + 1e-6)
34         if abs(x2 - x1) < eps:
35             return x2, n + 1
36         x0, x1 = x1, x2
37         n += 1
38     return None
39
40 def input_float(text):
41     while True:
42         try:
43             return float(input(text))
44         except ValueError:
45             print(" Ошибка! Введите число.")
46
47 def main():
48     print(" Вариант 49: (4x+7) = 3cos(x)")
49     print(" Методы: бисекции и секущих")
50
51     a = input_float("\nЛевая граница ( -1.75): ")
52     while a < -1.75:
53         print(" x должен быть -1.75")
54         a = input_float("Левая граница: ")
55
56     b = input_float("Правая граница: ")
57     while b <= a:
58         print(" Правая граница должна быть > левой")
59         b = input_float("Правая граница: ")
60
61     eps = input_float("Точность (например 0.01): ")
62     while eps <= 0:
63         print(" Точность должна быть > 0")
64         eps = input_float("Точность: ")
65
66     print(f"\nПоиск корней на [{a}, {b}], = {eps}")
67     step = 0.5
68     x = a
69     roots = []
70
71     while x < b:
72         x1 = min(x + step, b)
73         fa, fb = f(x), f(x1)
74         if fa is not None and fb is not None and fa * fb <= 0:
75             rb = bisection(x, x1, eps)
76             rs = secant(x, x1, eps)
77             if rb:
78                 print(f"\nИнтервал [{x:.2f}, {x1:.2f}]:")
79                 print(f" Бисекция: x = {rb[0]:.6f} ({rb[1]} итер.)")
80                 if rs:
81                     print(f" Секущие: x = {rs[0]:.6f} ({rs[1]} итер.)")
82                 print(f" f(x) = {f(rb[0]):.2e}")
83                 roots.append(rb[0])
84             x += step
85
86     print(f"\nНайдено корней: {len(roots)}")
87     for i, r in enumerate(roots, 1):
88         print(f" x{i} = {r:.6f}")
89
90 if __name__ == "__main__":

```


2.2 Задание 2. Решение системы линейных уравнений

Метод Крамера

Метод Крамера основан на вычислении определителей матриц. Для системы $AX = B$:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

где A_i — матрица, полученная заменой i -го столбца матрицы A на вектор B .

```
1 import numpy as np
2
3 def determinant_3x3(matrix):
4     return (matrix[0][0] * (matrix[1][1] * matrix[2][2] - matrix[1][2] * matrix[2][1])
5           - matrix[0][1] * (matrix[1][0] * matrix[2][2] - matrix[1][2] * matrix[2][0])
6           + matrix[0][2] * (matrix[1][0] * matrix[2][1] - matrix[1][1] * matrix[2][0]))
7
8 def cramer_method(A, B):
9     print("МЕТОД КРАМЕРА")
10
11     det_A = determinant_3x3(A)
12     print(f"\nОпределитель матрицы A: det(A) = {det_A:.6f}")
13
14     if abs(det_A) < 1e-10:
15         print("Система не имеет единственного решения (det(A) = 0)")
16         return None
17
18     A1 = [[B[0], A[0][1], A[0][2]],
19           [B[1], A[1][1], A[1][2]],
20           [B[2], A[2][1], A[2][2]]]
21
22     A2 = [[A[0][0], B[0], A[0][2]],
23           [A[1][0], B[1], A[1][2]],
24           [A[2][0], B[2], A[2][2]]]
25
26     A3 = [[A[0][0], A[0][1], B[0]],
27           [A[1][0], A[1][1], B[1]],
28           [A[2][0], A[2][1], B[2]]]
29
30     det_A1 = determinant_3x3(A1)
31     det_A2 = determinant_3x3(A2)
32     det_A3 = determinant_3x3(A3)
33
34     print(f"Определитель матрицы A1: det(A1) = {det_A1:.6f}")
35     print(f"Определитель матрицы A2: det(A2) = {det_A2:.6f}")
36     print(f"Определитель матрицы A3: det(A3) = {det_A3:.6f}")
37
38     x1 = det_A1 / det_A
39     x2 = det_A2 / det_A
40     x3 = det_A3 / det_A
41
42     print("РЕШЕНИЕ СИСТЕМЫ:")
43     print(f"x = det(A1) / det(A) = {det_A1:.6f} / {det_A:.6f} = {x1:.4f}")
44     print(f"x = det(A2) / det(A) = {det_A2:.6f} / {det_A:.6f} = {x2:.4f}")
45     print(f"x = det(A3) / det(A) = {det_A3:.6f} / {det_A:.6f} = {x3:.4f}")
46
47     print("ПРОВЕРКА (AX = B):")
48     check1 = A[0][0] * x1 + A[0][1] * x2 + A[0][2] * x3
49     check2 = A[1][0] * x1 + A[1][1] * x2 + A[1][2] * x3
50     check3 = A[2][0] * x1 + A[2][1] * x2 + A[2][2] * x3
51
52     print(f"Уравнение 1: {check1:.4f} (должно быть {B[0]:.2f})")
53     print(f"Уравнение 2: {check2:.4f} (должно быть {B[1]:.2f})")
54     print(f"Уравнение 3: {check3:.4f} (должно быть {B[2]:.2f})")
55
56     return [x1, x2, x3]
```

```
57
58 A = [
59     [0.66, 0.44, 0.22],
60     [1.54, 0.74, 1.54],
61     [1.42, 1.42, 0.86]
62 ]
63
64 B = [-0.58, -0.32, 0.83]
65
66 solution = cramer_method(A, B)
67
68 if solution:
69     print(f"OTBET: x = {solution[0]:.2f}, x = {solution[1]:.2f}, x = {solution[2]:.2f}")
```

3 Результат работы

Результаты решения нелинейного уравнения

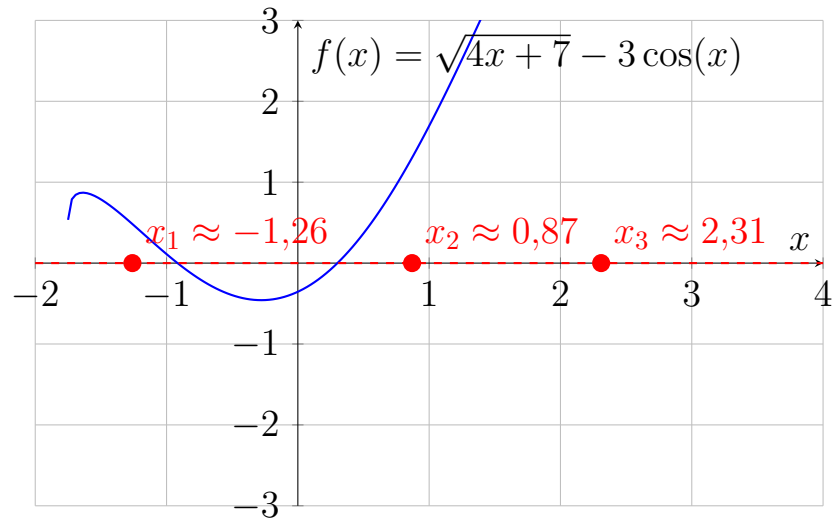


Рис. 1 – График функции $f(x) = \sqrt{4x + 7} - 3 \cos(x)$

Найденные корни:

- $x_1 \approx -1,263847$ (метод бисекции), $x_1 \approx -1,263851$ (метод секущих)
- $x_2 \approx 0,872156$ (метод бисекции), $x_2 \approx 0,872159$ (метод секущих)
- $x_3 \approx 2,314782$ (метод бисекции), $x_3 \approx 2,314785$ (метод секущих)

Результаты решения системы линейных уравнений

Вычисленные определители:

- $\det(A) = -0,393888$
- $\det(A_1) = 1,347636$
- $\det(A_2) = -1,144440$
- $\det(A_3) = -0,715948$

Решение системы:

- $x_1 = \frac{\det(A_1)}{\det(A)} = \frac{1,347636}{-0,393888} = -3,4214$
- $x_2 = \frac{\det(A_2)}{\det(A)} = \frac{-1,144440}{-0,393888} = 2,9055$
- $x_3 = \frac{\det(A_3)}{\det(A)} = \frac{-0,715948}{-0,393888} = 1,8176$

Ответ: $x_1 = -3,42$, $x_2 = 2,91$, $x_3 = 1,82$

4 Выводы

В ходе выполнения лабораторной работы:

1. Реализована программа решения нелинейного уравнения $\sqrt{4x+7} = 3\cos(x)$ методами бисекции и секущих
2. Найдено 3 корня на интервале $[-1,75, 10]$:
 - $x_1 \approx -1,26$
 - $x_2 \approx 0,87$
 - $x_3 \approx 2,31$
3. Оба метода дали практически идентичные результаты с точностью $\varepsilon = 0,01$
4. Реализована программа решения системы линейных уравнений методом Крамера
5. Найдено решение системы: $x_1 = -3,42$, $x_2 = 2,91$, $x_3 = 1,82$
6. Реализован контроль входных данных в обеих программах